



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 5: Reinforcement Learning

Dr Pedro Machado pedro.machado@ntu.ac.uk



Nottingham Trent
University

Department of Computer Science

Lecture Outline

In this lecture will cover:

- Reinforcement learning for sequential decision-making
- Agent-environment interaction and MDPs
- States, actions, rewards, and return
- Policies and value functions
- Bellman equations
- Exploration vs exploitation
- Model-free learning (Q-learning, SARSA)
- Deep reinforcement learning
- Policy gradients and actor-critic methods
- RL for control and CI parameter tuning

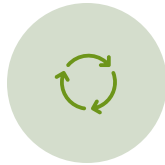
This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.

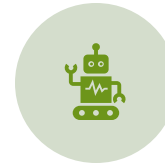
Why Reinforcement Learning for Control and Optimisation



Many real-world control and optimisation problems involve sequential decisions under uncertainty, where actions influence future system states and long-term performance.



Classical control and optimisation methods often rely on accurate mathematical models, which may be unavailable, incomplete, or costly to derive for complex or nonlinear systems.



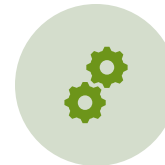
Reinforcement learning enables data-driven optimisation by learning control policies directly through interaction with the environment, without requiring an explicit system model.



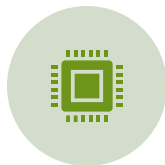
Long-term objectives can be optimised naturally, rather than myopic or single-step performance criteria, through cumulative reward maximisation.



Adaptive behaviour can be achieved in non-stationary or changing environments, supporting self-tuning and online optimisation.



Complex constraints and trade-offs can be incorporated through reward design, enabling multi-objective optimisation within a unified framework.

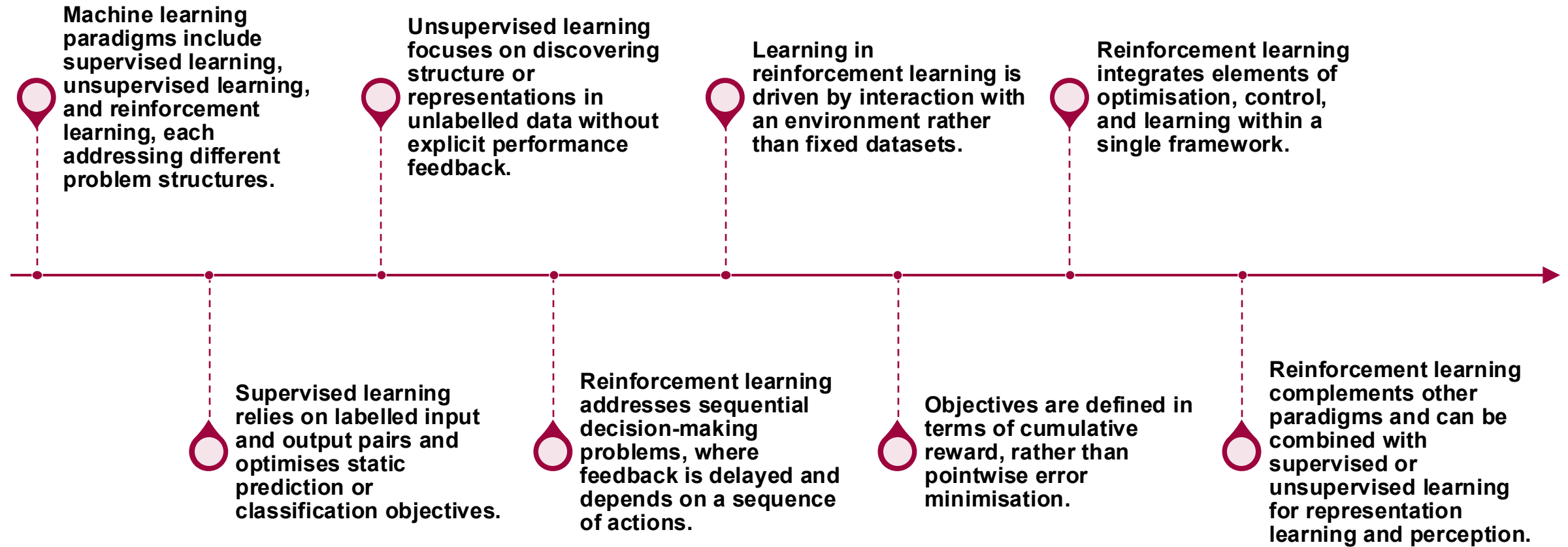


High-dimensional state and action spaces, common in modern control and CI systems, can be handled using function approximation and deep learning.



Reinforcement learning provides a unifying framework connecting control theory, optimisation, and computational intelligence, supporting scalable and autonomous decision-making systems.

Positioning Reinforcement Learning within ML Paradigms



Sequential Decision-Making Problems

Decisions are made over time, where each action influences future states of the system.

Outcomes depend on sequences of actions rather than isolated decisions.

Feedback is often delayed, sparse, or indirect, making optimisation challenging.

Uncertainty may arise from stochastic environments, partial observability, or unknown dynamics.

Objectives are typically long-term, requiring trade-offs between immediate and future performance.

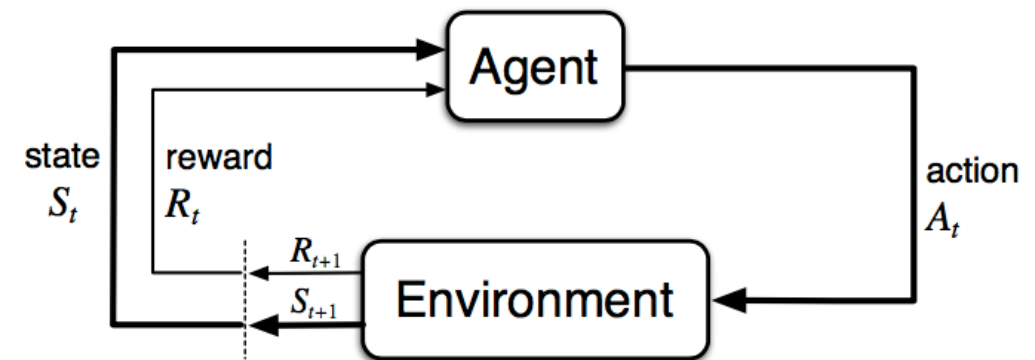
Agent-Environment Interaction Loop



- An agent observes the current state of the environment.
- The agent selects an action according to its policy.
- The environment transitions to a new state based on the action taken.
- A reward signal is returned to the agent, reflecting the quality of the action.
- The agent updates its internal knowledge or policy using the observed experience.
- Interaction is repeated over time, forming a sequence of state–action–reward transitions.
- Learning emerges from trial-and-error interaction rather than explicit supervision.
- The objective is to learn a policy that maximises long-term cumulative reward.

Markov Decision Processes (MDP): intuition

- A mathematical framework for modelling sequential decision-making under uncertainty.
- System evolution depends on the current state and chosen action, not on past history.
- Future outcomes are probabilistic rather than deterministic.
- Decisions influence both immediate rewards and future possibilities.
- The environment provides feedback through state transitions and rewards.
- Long-term performance is prioritised over single-step gains.
- Control and optimisation problems can be expressed in a unified, principled way.
- Simplifies complex dynamics while retaining essential decision-making structure.

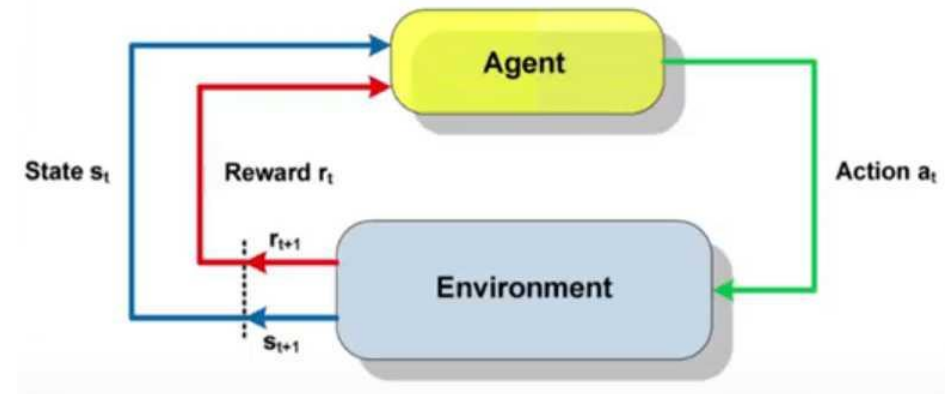


Reinforcement Learning for Optimisation

- RL is a learning paradigm where an **agent** interacts with an **environment**, taking actions to maximise cumulative **reward**. Unlike supervised learning, RL **learns from experience** rather than labelled data.

Key Components:

- **Agent:** Learner or decision-maker.
- **Environment:** Everything the agent interacts with.
- **State (s):** Current situation.
- **Action (a):** Decision taken by the agent.
- **Reward (r):** Feedback signal from the environment.



Applications in CI:

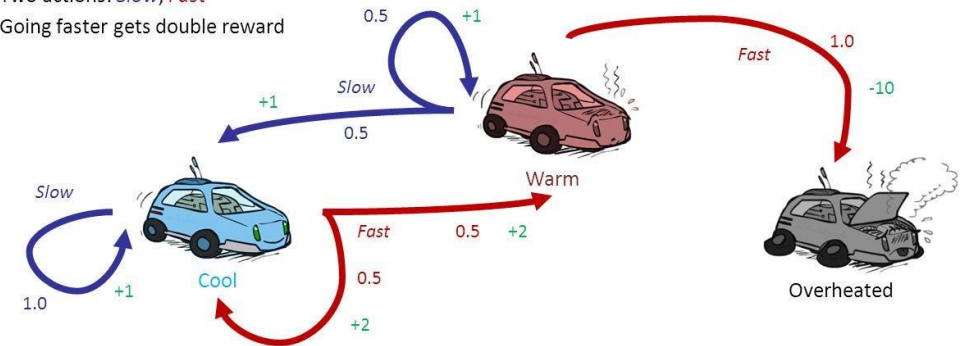
- Tuning hyperparameters of evolutionary or swarm algorithms.
- Dynamic resource allocation and scheduling.
- Adaptive control systems and real-time decision-making.

States, actions, rewards in control systems

- States represent the measurable or estimated condition of a system.
- Actions correspond to control inputs or parameter changes.
- Rewards encode performance objectives and constraints.
- State transitions reflect system dynamics.
- Rewards provide scalar feedback on control quality.
- Both continuous and discrete systems can be modelled.
- Control problems map naturally to RL formulations.
- Reward design links engineering goals to learning.

Example: Racing

- A robot car wants to travel far, quickly
- Three states: *Cool*, *Warm*, *Overheated*
- Two actions: *Slow*, *Fast*
- Going faster gets double reward



Return and discounted cumulative reward



THE RETURN IS THE
TOTAL ACCUMULATED
REWARD OVER TIME.



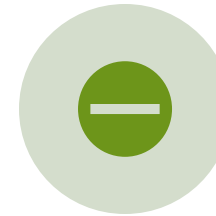
FUTURE REWARDS
ARE DISCOUNTED TO
REFLECT
UNCERTAINTY.



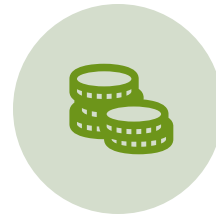
THE DISCOUNT
FACTOR BALANCES
SHORT-TERM AND
LONG-TERM
OBJECTIVES.



HIGH DISCOUNTING
PRIORITISES
IMMEDIATE
PERFORMANCE.



LOW DISCOUNTING
FAVOURS LONG-TERM
OPTIMISATION.



DISCOUNTING
ENSURES
MATHEMATICAL
CONVERGENCE.

Value functions: state-value and action-value

The state-value function $V(s)$ measures the expected long-term return starting from a given state.

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right]$$

Variables

- $V^\pi(s)$: Expected long-term return when starting in state s and following policy π
- $\mathbb{E}_\pi[\cdot]$: Expectation over trajectories generated by policy π
- s : Current state
- t : Time step index
- r_{t+1} : Reward received after time step t
- γ : Discount factor ($0 \leq \gamma \leq 1$), weighting future rewards
- π : Policy defining action selection
- s_0 : Initial state

Bellman expectation equations

- Bellman equations express value functions recursively.
- The value of a state depends on immediate reward and future state values.

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s, a_0 = a \right]$$

Variables

- $Q^{\pi}(s, a)$: Expected long-term return starting from state s , taking action a , then following policy π
- a : Action taken in state s
- Other variables: same meaning as above

Bellman optimality equations

- Bellman optimality equations define the best achievable value functions.
- Optimal value is obtained by maximising over all actions.
- They characterise the optimal policy implicitly.

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Variables

- $V^\pi(s)$: Value of state s under policy π
- $\pi(a|s)$: Probability of selecting action a in state s
- $P(s'|s, a)$: Probability of transitioning to state s' after action a in state s
- s' : Next state
- $R(s, a, s')$: Immediate reward for transition $s \rightarrow s'$ via action a
- γ : Discount factor

Policy definition and representation

- A policy defines how actions are selected in each state.
- It maps states to actions or action probabilities.
- Policies encode the agent's decision-making strategy.

$$\pi(a|s) = \Pr(A_t = a \mid S_t = s)$$

Variables

- $\pi(a|s)$: Probability of choosing action a in state s
- A_t : Action at time step t
- S_t : State at time step t

Deterministic vs stochastic policies

Aspect	Deterministic Policies	Stochastic Policies
Action selection	Select a single action for each state	Select actions according to a probability distribution
Response to uncertainty	Less robust to uncertainty and noise	More robust to uncertainty and variability
Exploration behaviour	No inherent exploration	Natural exploration through randomness
Learning implications	Simpler to analyse and deploy	Better suited for exploration during learning
Typical use cases	Final control policies, deployment	Learning phases, uncertain environments

Exploration vs exploitation dilemma

Aspect	Exploitation	Exploration
Action choice	Selects actions known to perform well	Tests actions with uncertain outcomes
Objective	Maximise immediate or known performance	Improve knowledge about the environment
Risk profile	Low short-term risk	Higher short-term risk
Long-term effect	May converge to suboptimal solutions	Enables discovery of better solutions
Impact on performance	Stable short-term performance	Temporary performance degradation
Role in learning	Uses existing knowledge	Acquires new knowledge
Control system concern	Maintains operational performance	Must be carefully constrained for safety
Effect on efficiency	High when optimal actions are known	Improves learning efficiency when balanced

ϵ -greedy and soft exploration strategies

- ϵ -greedy randomly selects actions with probability ϵ .
- Greedy actions are chosen with probability $1 - \epsilon$.
- ϵ controls the exploration rate.
- Softmax assigns probabilities based on value estimates.

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|}, & a = \arg \max_a Q(s, a) \\ \frac{\epsilon}{|A|}, & \text{otherwise} \end{cases}$$

Variables

- ϵ : Exploration rate
- $|A|$: Number of available actions
- $\arg \max_a Q(s, a)$: Action with highest estimated value

Softmax exploration

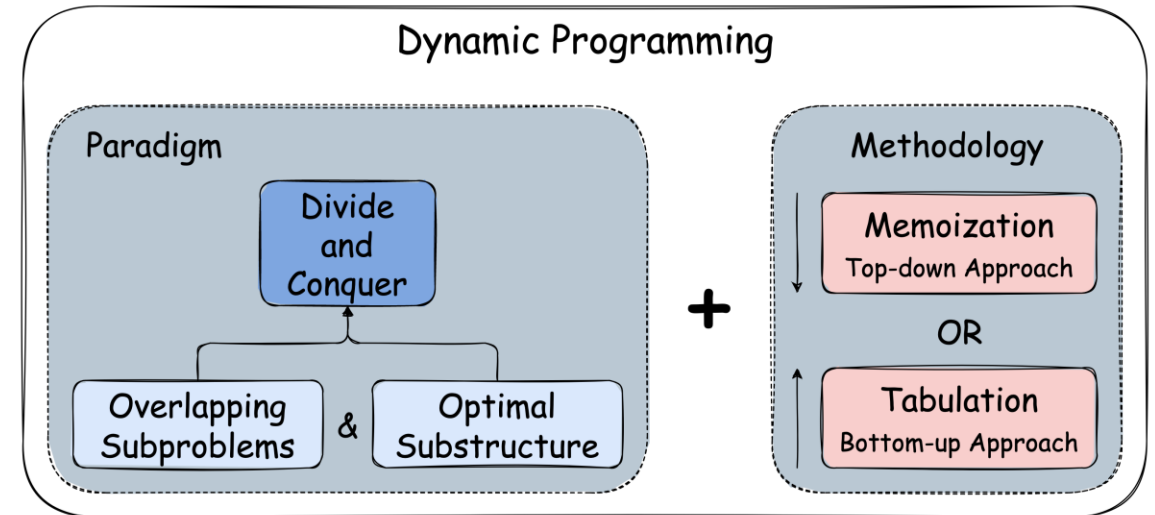
$$\pi(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

Variables

- τ : Temperature parameter controlling exploration level
- e : Exponential function

Dynamic programming overview

- Dynamic programming assumes a known environment model.
- It computes exact value functions through recursion.
- Full state-space sweeps are required.
- Policy evaluation and improvement are alternated.
- Provides strong theoretical guarantees.
- Computational cost grows rapidly with problem size.

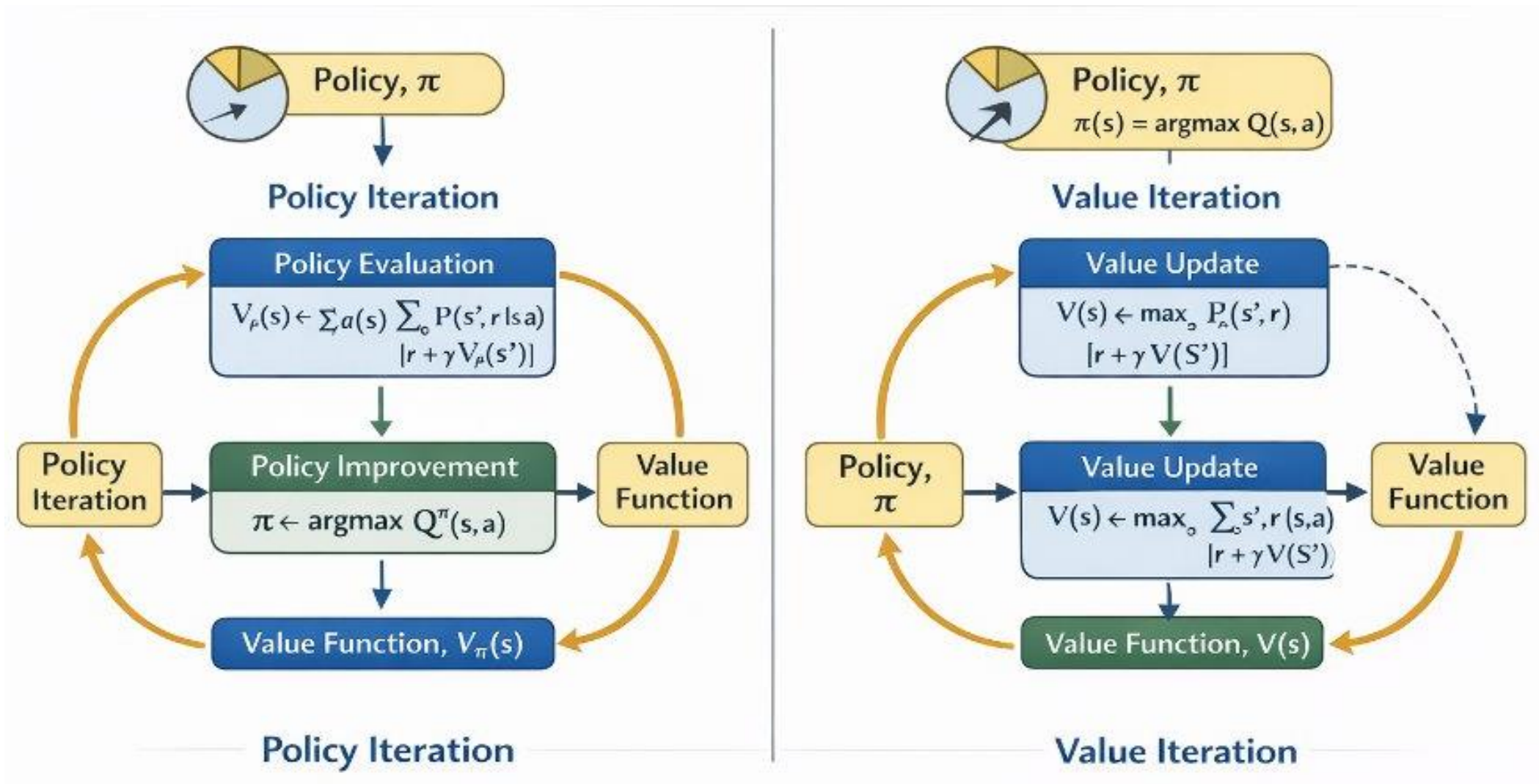


Policy evaluation and policy improvement

- **Policy evaluation** estimates value functions for a fixed policy.
- It quantifies expected long-term performance.
- **Policy improvement** updates the policy using value estimates.
- Actions with higher value are favoured.

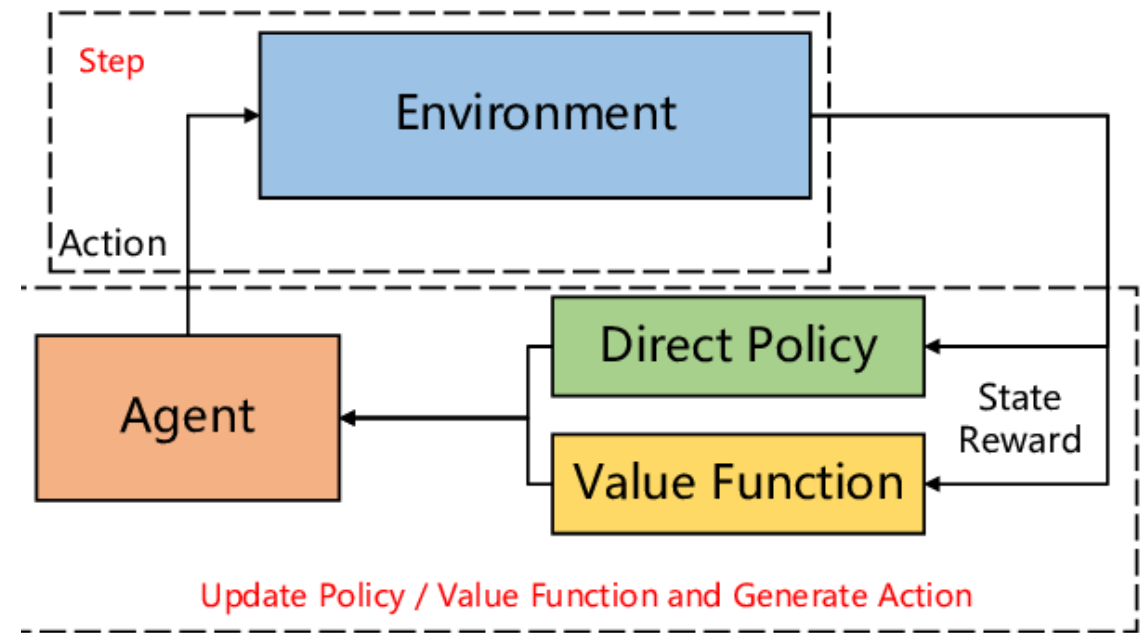


Policy iteration and value iteration



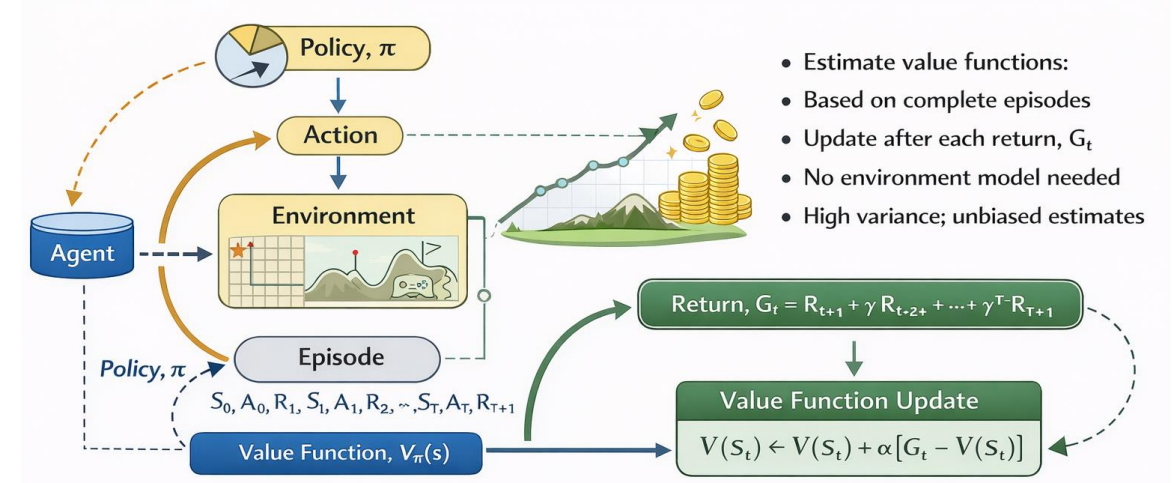
Model-free reinforcement learning

- Model-free methods do not require environment dynamics.
- Learning occurs directly from experience.
- Suitable for unknown or complex systems.



Monte Carlo methods

- Monte Carlo methods learn from complete episodes.
- Returns are computed after episode termination.
- No bootstrapping is used.
- Estimates are unbiased.
- Variance can be very high.
- Not suitable for continuing tasks.



Temporal Difference (TD) learning

- TD learning updates values incrementally.
- Learning occurs before episode termination

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Variables

- s_t : Current state
- s_{t+1} : Next state
- α : Learning rate
- Term in brackets: **TD error**

TD(0) and bootstrapping intuition

- TD(0) performs one-step value updates.
- Targets use current value estimates.
- Learning occurs at every time step.

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Variables

- δ_t : Temporal Difference error
- Measures prediction error of current value estimate

Q-learning algorithm

- Q-learning learns action-value functions directly.
- It is an off-policy algorithm.
- Learning targets use greedy action selection.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Variables

- a_t : Action taken at time t
- $\max_a Q(s_{t+1}, a)$: Best estimated future value

SARSA and on-policy learning

- SARSA learns values following the current policy.
- It is an on-policy algorithm.
- Updates reflect actual executed actions.
- Learning incorporates exploration behaviour.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Variables

- a_{t+1} : Next action chosen by the current policy

Comparison: Q-learning vs SARSA

Aspect	Q-learning	SARSA
Policy type	Off-policy learning	On-policy learning
Action used for update	Greedy action (max Q)	Action actually taken by the policy
Learning behaviour	Optimistic learning	Conservative learning
Treatment of exploration	Ignores exploration in updates	Incorporates exploration behaviour
Stability under exploration	Can be less stable with high exploration	More stable during exploration
Behaviour during learning	More aggressive action updates	More cautious action updates
Practical safety	May take risky actions	Often safer in practice
Typical use cases	Faster convergence in simulation	Safer learning in real-world control

Function approximation in RL

- Large state spaces make tabular methods infeasible.
- Function approximation generalises across states.
- Values or policies are approximated.

$$Q(s, a; \theta) \approx Q^*(s, a)$$

Variables

- θ : Parameters of the function approximator (e.g. neural network)

Linear vs non-linear approximators

Aspect	Linear Approximators	Non-linear Approximators
Model structure	Linear combination of features	Non-linear function mappings
Expressiveness	Limited representational power	High representational power
Stability	Generally stable during learning	Higher risk of instability
Analytical tractability	Easy to analyse theoretically	Difficult to analyse formally
Ability to capture complexity	Suitable for simple relationships	Captures complex, nonlinear patterns
Bias–variance trade-off	Higher bias, lower variance	Lower bias, higher variance
Computational cost	Low computational overhead	Higher computational cost
Typical use cases	Small or well-structured problems	High-dimensional, complex tasks

Deep Reinforcement Learning motivation



Real problems involve high-dimensional observations.



System dynamics are often nonlinear.



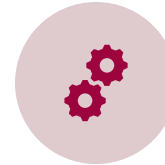
Tabular methods do not scale.



Deep learning provides representation learning.



Combines perception and control.



Enables end-to-end optimisation.



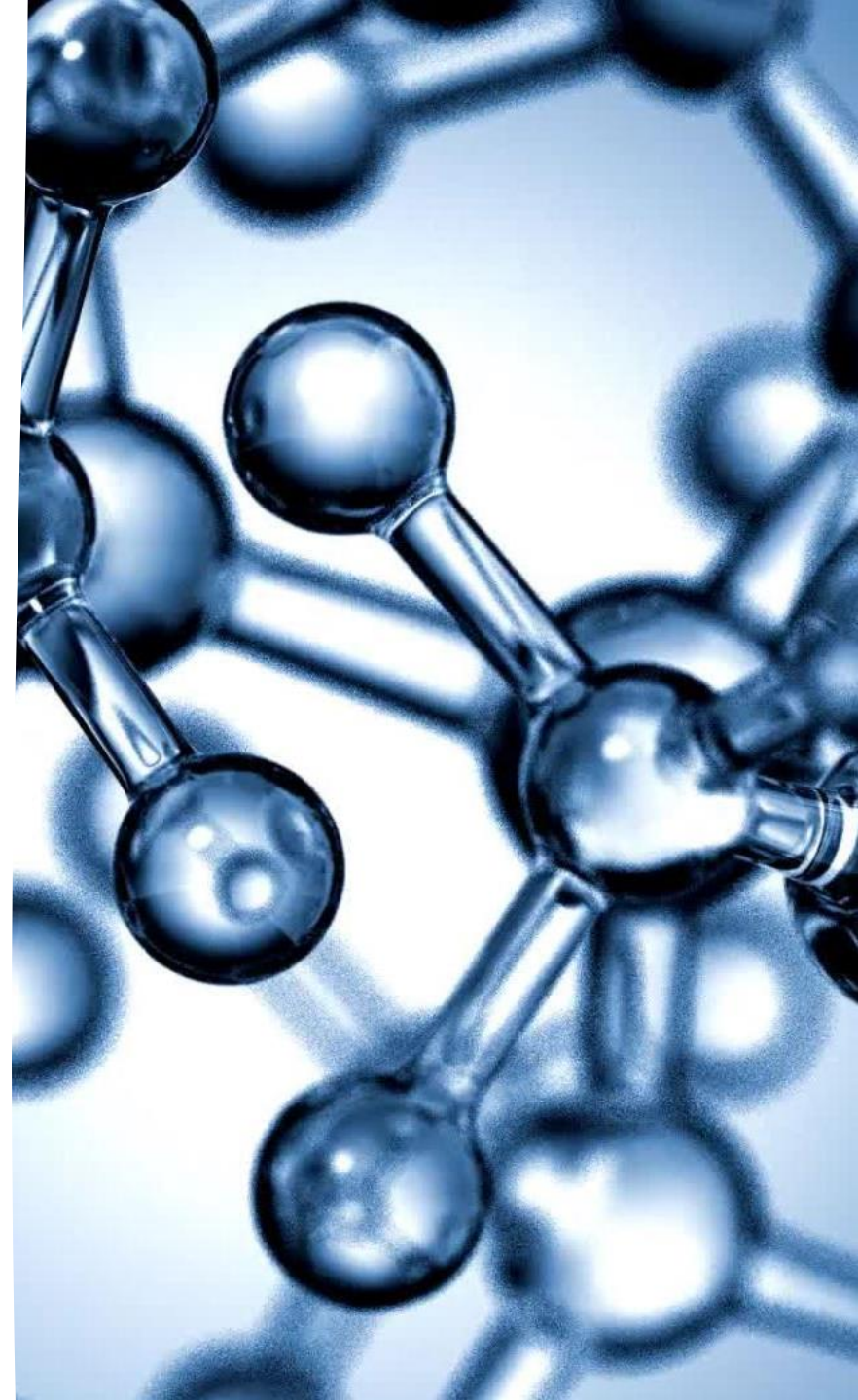
Has achieved major empirical success.



Introduces new stability challenges.

Neural networks as value function approximators

- Neural networks approximate $V(s)$ or $Q(s, a)$.
- Raw observations can be used as input.
- Features are learned automatically.
- Generalisation across states is achieved.
- Enables learning from high-dimensional data.
- Training uses gradient-based methods.
- Stability requires careful design.
- Foundation of deep RL.



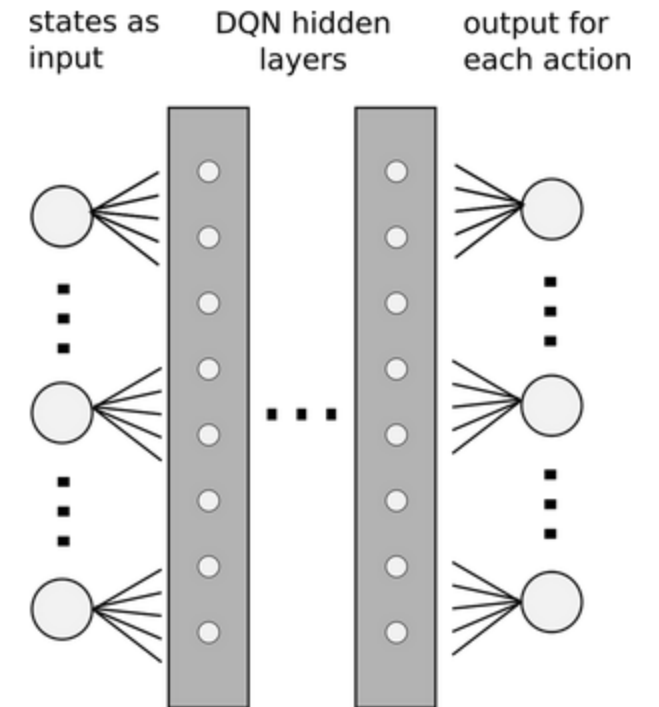
Deep Q-Network (DQN) architecture

- DQN uses a neural network to estimate Q-values.
- Inputs represent system states.
- Outputs correspond to action values.
- Training uses TD learning.

$$L(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2$$

Variables

- $L(\theta)$: Training loss
- θ^- : Target network parameters



Experience replay and target networks

Experience replay stores past transitions.

Random sampling breaks data correlation.

Improves learning stability.

Target networks stabilise learning targets.

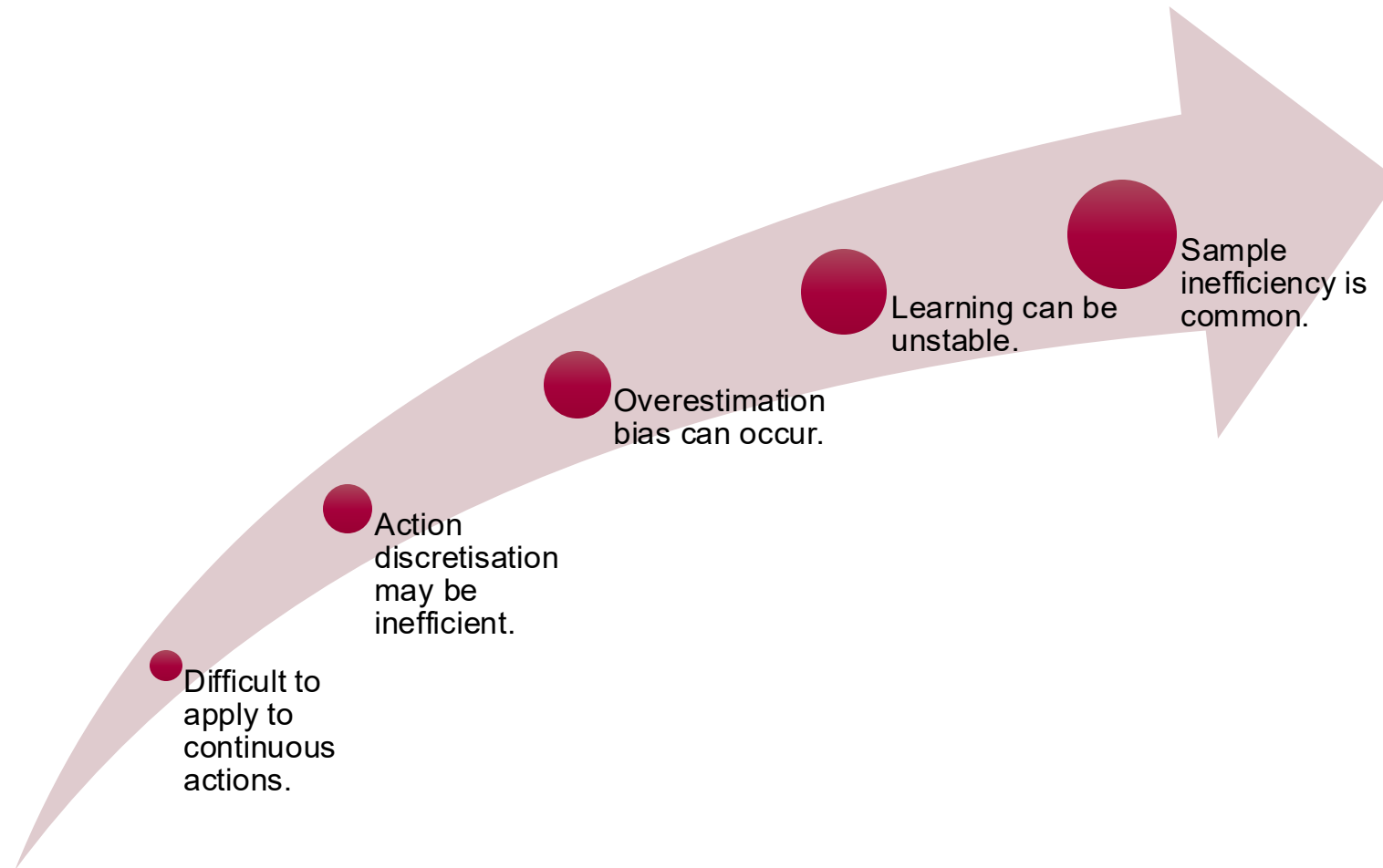
Reduce oscillations and divergence.

Essential for deep RL.

Improve convergence behaviour.

Widely adopted techniques.

Limitations of value-based methods



Policy gradient methods

- Policy gradients optimise policies directly.
- Avoid explicit value maximisation.
- Naturally handle continuous actions.

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a)]$$

Variables

- $J(\theta)$: Expected return objective
- ∇_{θ} : Gradient w.r.t. parameters θ

REINFORCE algorithm intuition

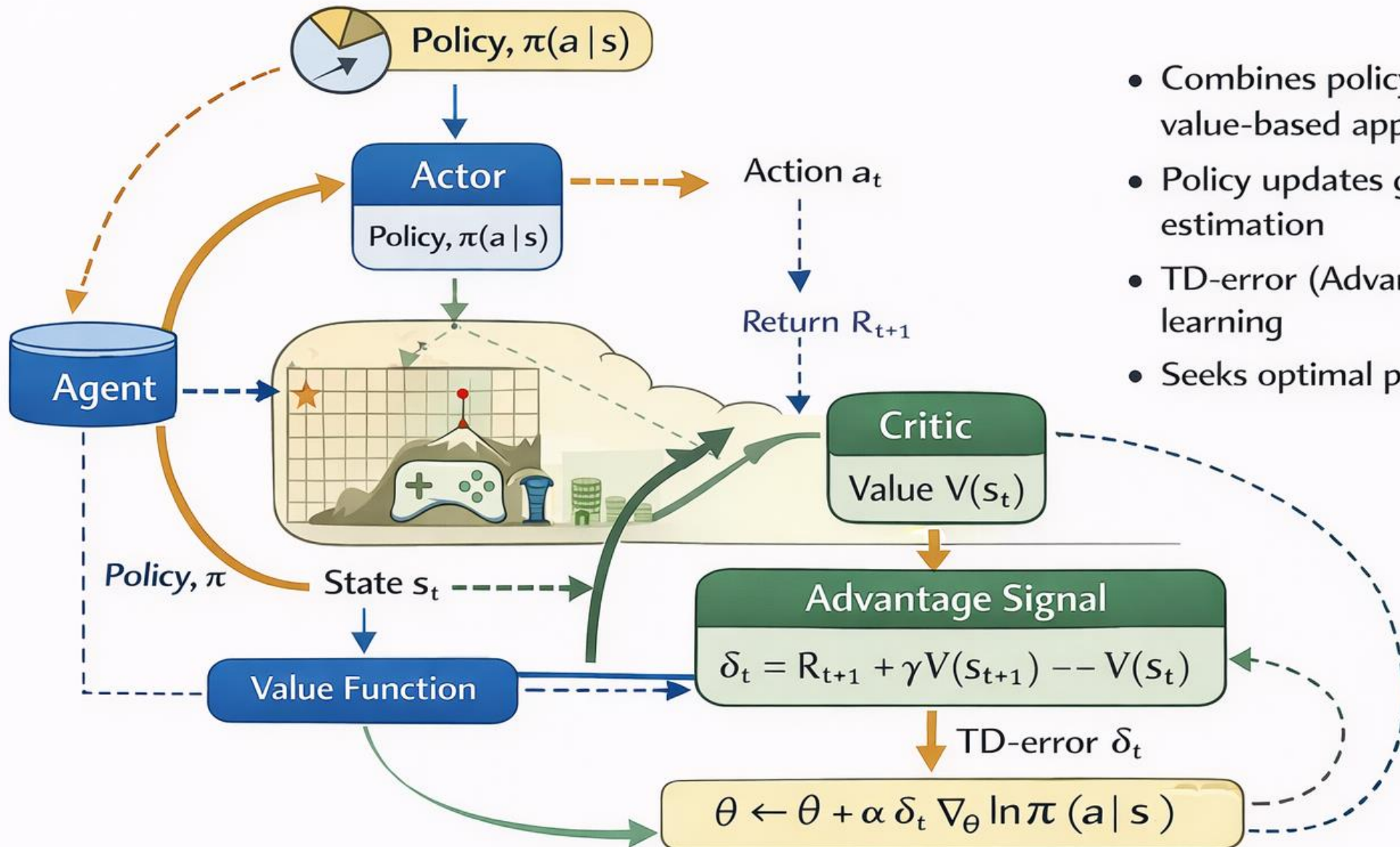
- REINFORCE uses Monte Carlo returns.
- Gradients are weighted by rewards.
- Updates increase probability of good actions.

$$\Delta\theta \propto \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) G_t$$

Variables

- G_t : Return following time t

Actor-Critic Architectures



- Combines policy-based and value-based approaches
- Policy updates guided by value estimation
- TD-error (Advantage) drives learning
- Seeks optimal policy π^*

Advantage functions and variance reduction

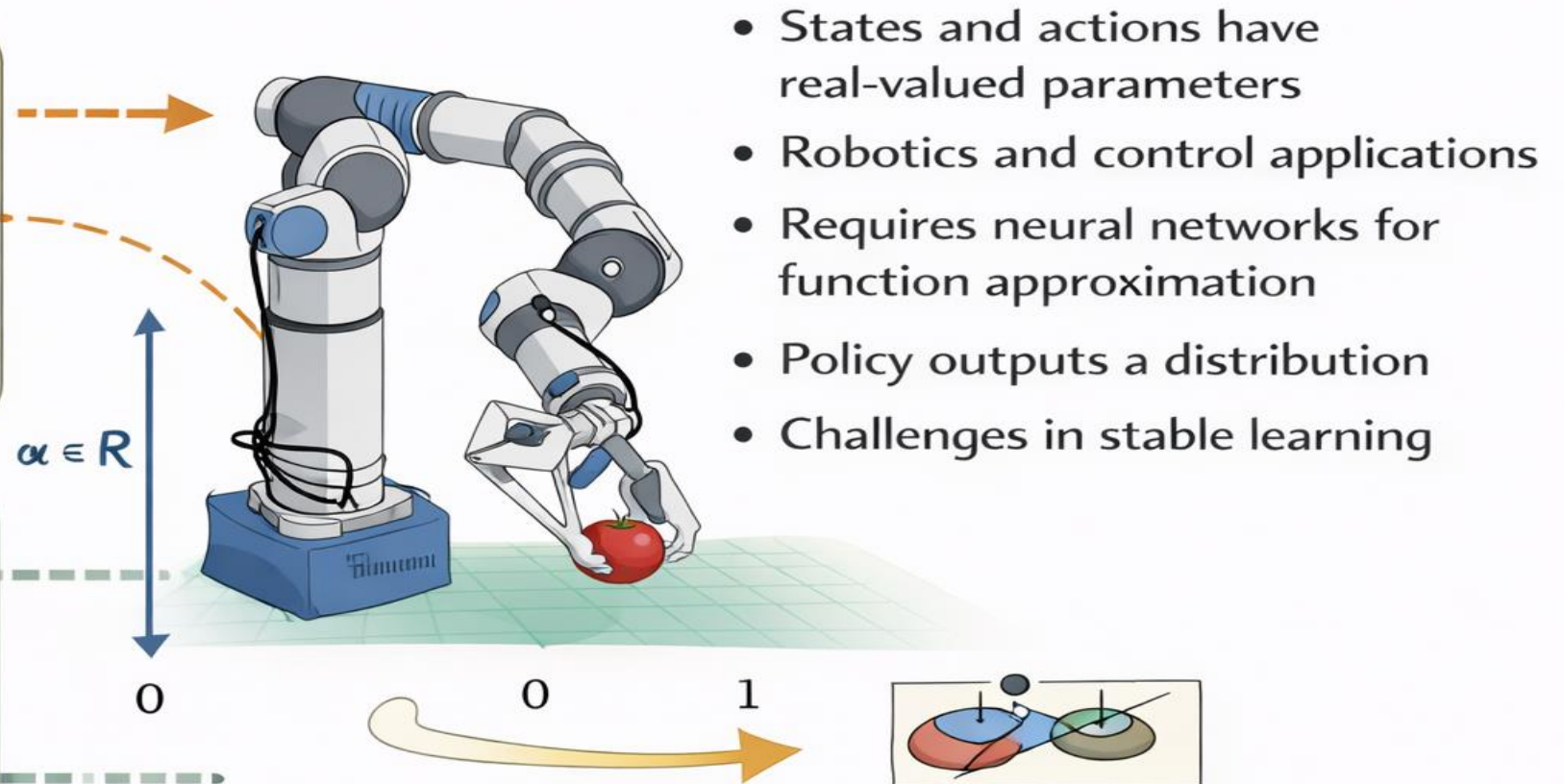
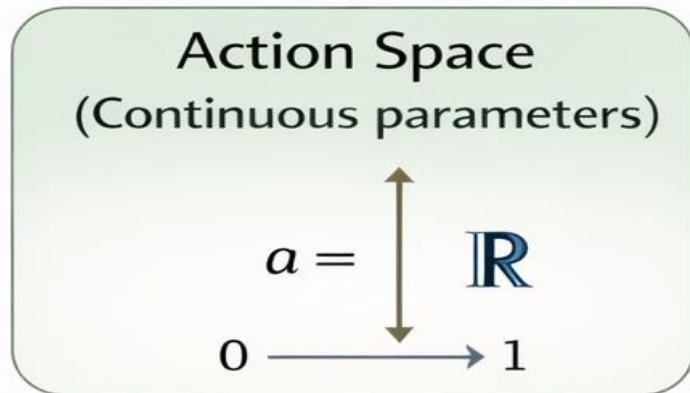
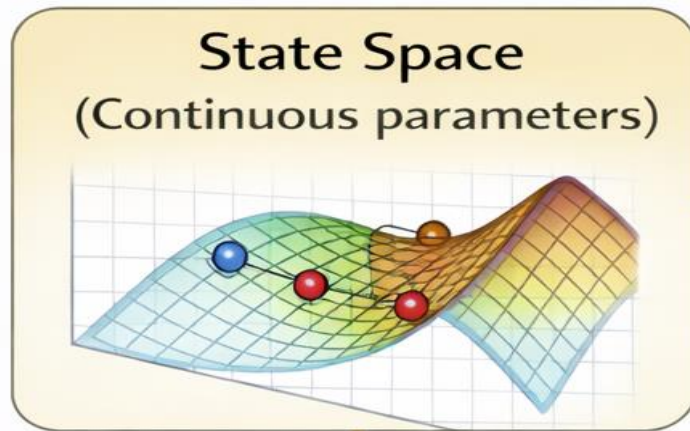
- Advantage measures relative action quality.
- Compares action value to baseline.
- Removes irrelevant reward components.
- Reduces gradient variance.

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

Variables

- $A^{\pi}(s, a)$: Advantage of action a over average behaviour

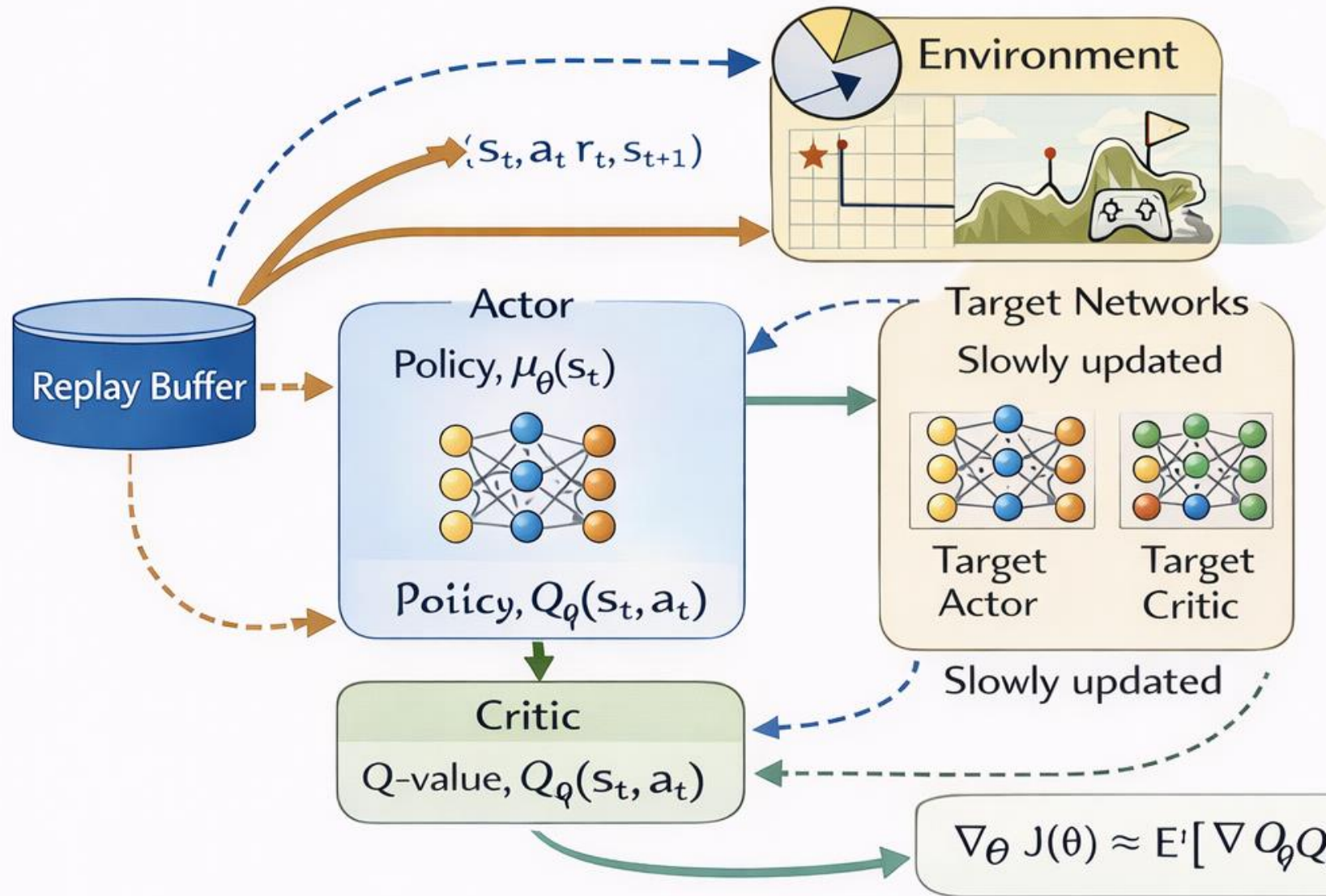
Continuous Control and Action Spaces



$$\pi(w | s) \sim N(\mu(s, \Sigma_a) = E\{ \mathbf{a} | s \})$$

$$\nabla_{\theta} J(\theta) = E[\delta_{\theta} \log \pi(s, a)]$$

Deterministic Policy Gradients (DDPG)



- Combines DDPG and DQN principles
- Actor learns deterministic policy $\mu_\theta(s)$
- Critic learns value function $Q_\phi(s, a)$
- Experience replay stabilizes training
- Target networks improve stability

Proximal Policy Optimisation (PPO)

- PPO limits policy update size.
- Prevents destructive updates.
- Improves training stability.

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

Variables

- $r_t(\theta)$: Probability ratio
- θ_{old} : Parameters before update

Stability and sample efficiency issues

Deep RL training can be unstable.

Large data requirements are common.

Hyperparameter sensitivity is high.

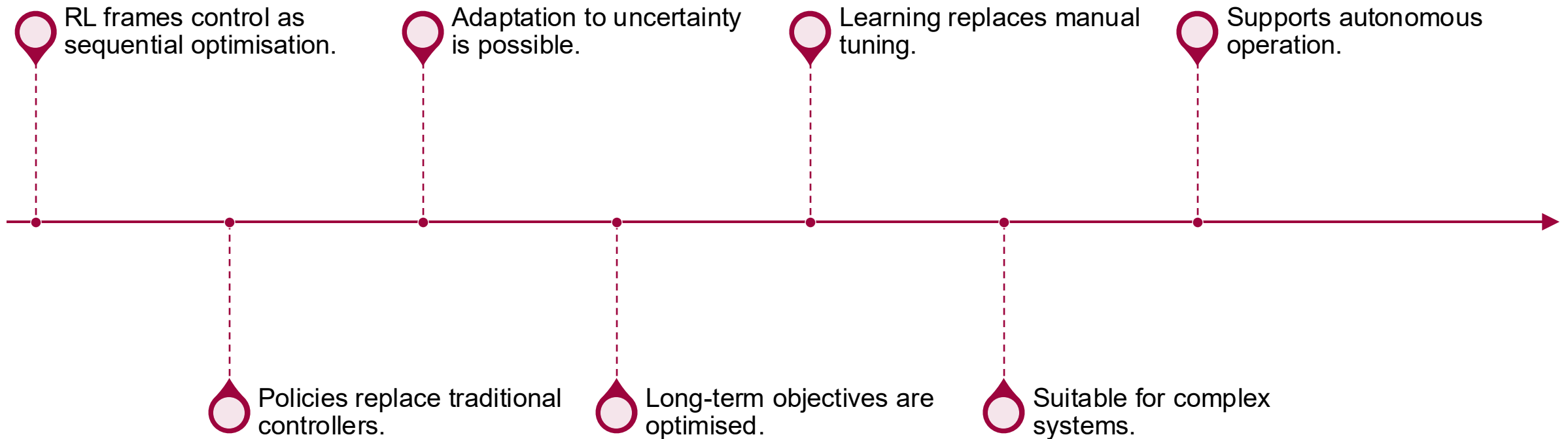
Exploration can be unsafe.

Sample efficiency is critical in control.

Simulation is often required.

Stability limits deployment.

RL for control and optimisation problems



Classical control vs RL-based control

Aspect	Classical Control	RL-based Control
System knowledge	Requires explicit system model	Learns directly from data
Treatment of dynamics	Based on known or identified dynamics	Dynamics learned implicitly
Theoretical guarantees	Strong stability and performance guarantees	Limited or empirical guarantees
Adaptability	Limited adaptation once designed	High adaptability to changing conditions
Handling model uncertainty	Sensitive to modelling errors	More robust to model uncertainty
Design effort	High analytical and modelling effort	Higher data and training effort
Integration trend	Increasingly combined with learning methods	Often integrated with classical control
Suitability	Well-understood, structured systems	Complex or poorly modelled systems

Constraints, safety, and stability considerations

- Control systems may be safety-critical.
- Constraints must be respected during learning.
- Unsafe exploration is unacceptable.
- Stability must be maintained.
- Risk-aware methods are required.
- Constraints may be embedded in rewards.
- Safe RL is an active research area.
- Deployment requires caution.



RL in adaptive and self-tuning controllers



Controllers adapt online.



Non-stationary environments are handled.



Performance degrades less over time.



Manual retuning is reduced.



Learning responds to system changes.



Supports autonomous operation.



Suitable for long-term deployment.

Multi-objective optimisation with RL



Real systems have competing objectives.



Rewards may combine multiple criteria.



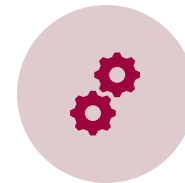
Weighting balances trade-offs.



Pareto-optimal solutions are possible.



Flexibility in decision-making.



Supports complex optimisation goals.



Requires careful design.

Simulation-to-real transfer challenges

- Simulations simplify real systems.
- Learned policies may fail in reality.
- Model mismatch causes errors.
- Robust policies are required.
- Domain randomisation is common.
- Transfer remains challenging.
- Safety is a concern.



Sample efficiency and data limitations

Real-world data
is expensive.

Learning speed
is critical.

Sample-
inefficient
methods are
impractical.

Prior knowledge
may be
leveraged.

Model-based
methods help
efficiency.

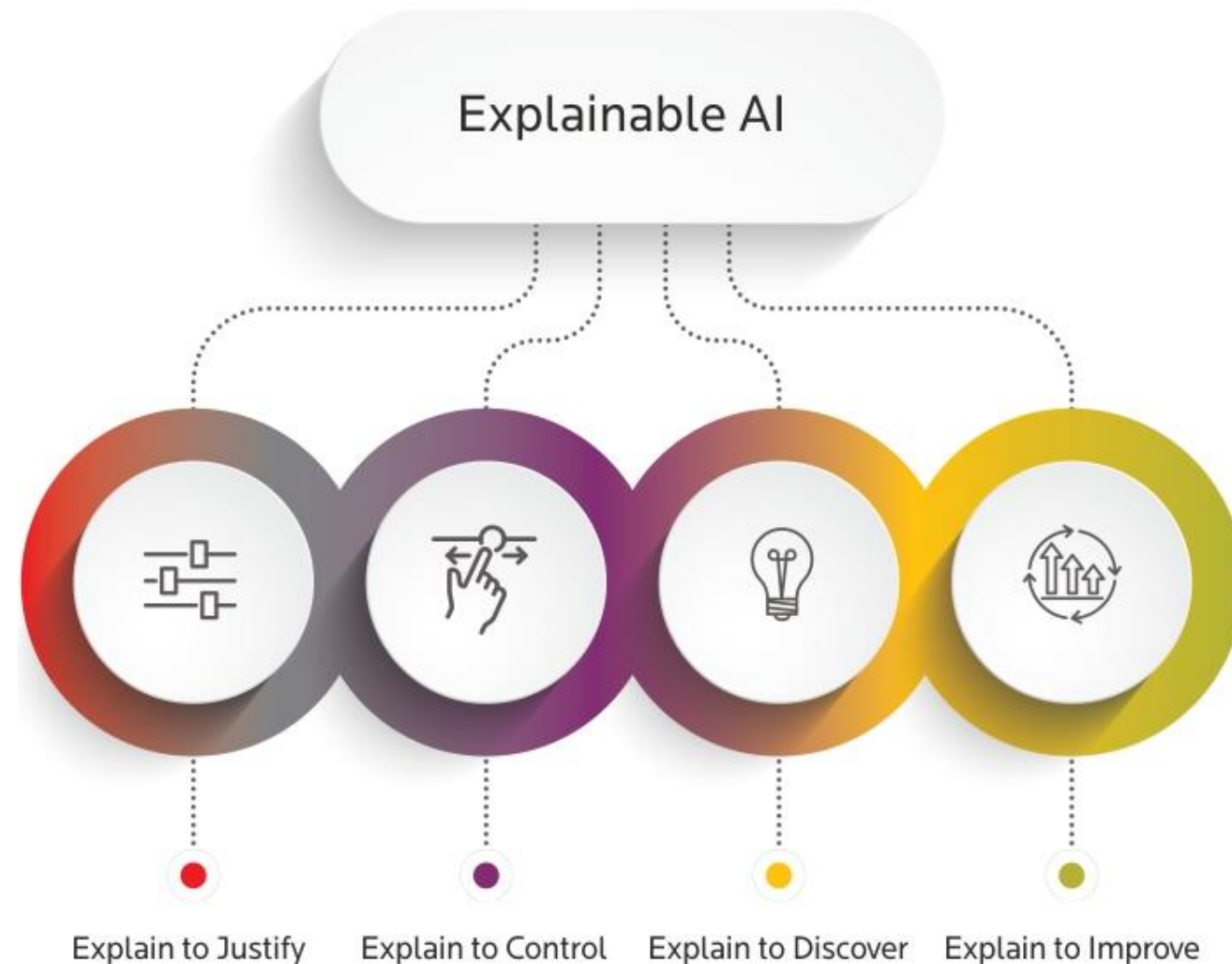
Hybrid
approaches are
emerging.

Data limits
deployment.

Key challenge in
control.

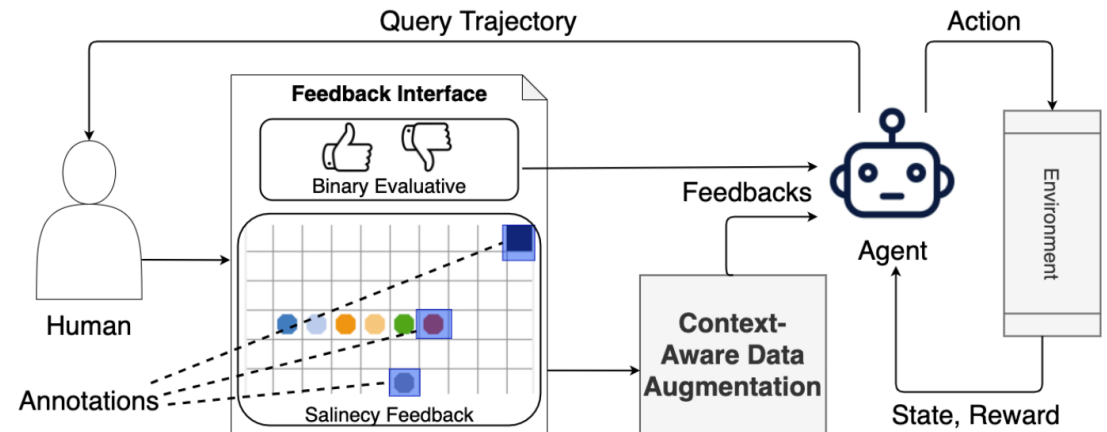
Explainability and trust in RL controllers

- Learned policies may be opaque.
- Understanding decisions is difficult.
- Trust is required for deployment.
- Explainability aids debugging.
- Validation is essential.
- Regulatory concerns may arise.
- Interpretability supports acceptance.
- Important for safety-critical systems.



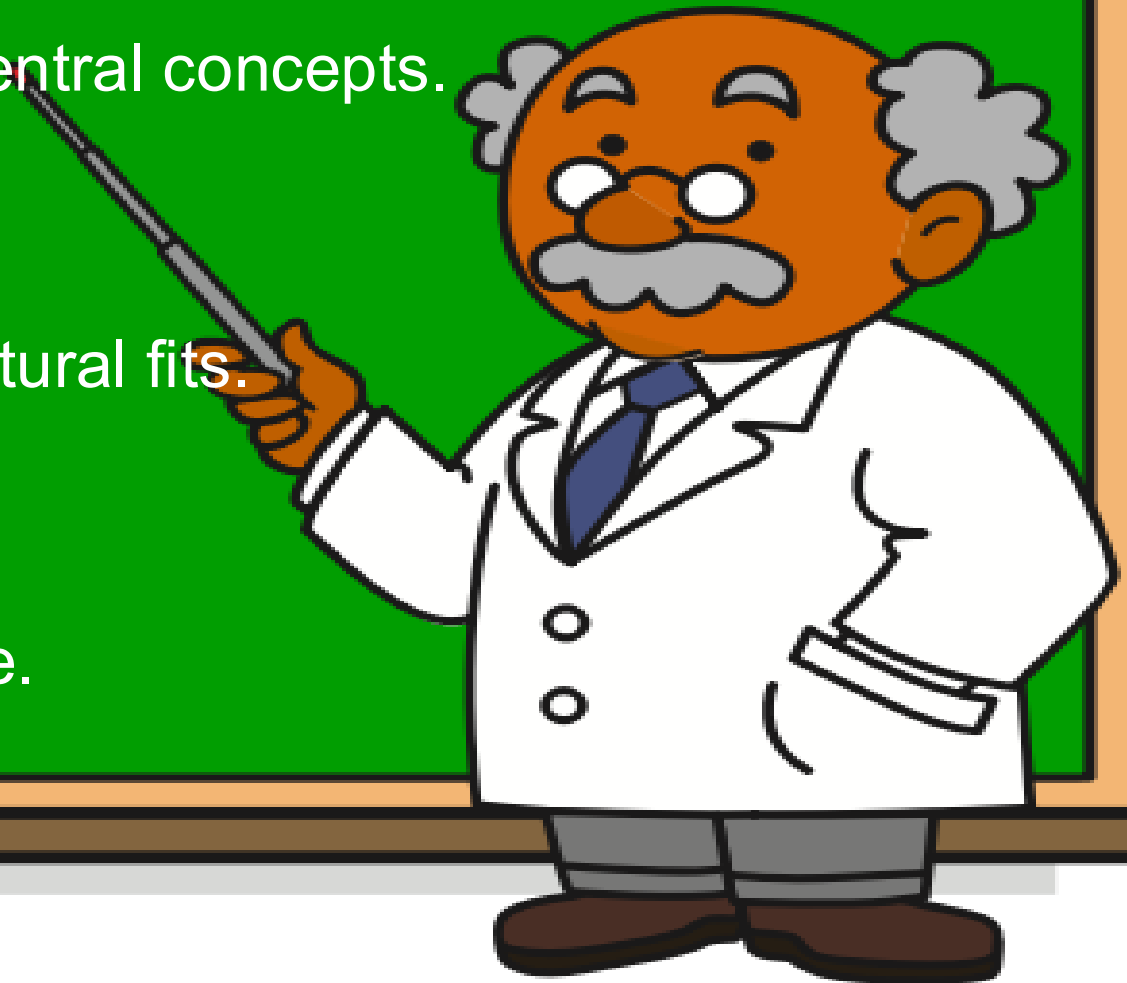
Human-in-the-loop and safe RL

- Human expertise can guide learning.
- Feedback improves safety.
- Exploration can be constrained.
- Shared control improves reliability.
- Learning is accelerated.
- Humans supervise critical decisions.
- Combines automation and oversight.



Key takeaways

- RL provides a framework for sequential optimisation.
- Value functions and policies are central concepts.
- Multiple algorithm families exist.
- Deep RL enables scalability.
- Control and CI applications are natural fits.
- Reward design is critical.
- Safety and stability matter.
- Practical deployment requires care.



Quiz (1/3)

Q1. Which statement best describes the goal of reinforcement learning?

- A. To minimise prediction error on labelled datasets
- B. To discover hidden structure in unlabelled data
- C. To maximise cumulative reward through sequential decision-making
- D. To optimise a single-step objective function

Q2. What is the main difference between the state-value function $V(s)$ and the action-value function $Q(s,a)$?

- A. $V(s)$ includes future rewards, while $Q(s,a)$ does not
- B. $V(s)$ depends on the current state only, while $Q(s,a)$ depends on state and action
- C. $Q(s,a)$ applies only to deterministic policies
- D. $V(s)$ is used only in deep reinforcement learning

Quiz (2/3)

Q3. What does the maximisation operator in the Bellman optimality equation represent?

- A. Averaging over all possible actions
- B. Selecting the action that minimises immediate cost
- C. Choosing the action that maximises expected long-term return
- D. Sampling actions according to the policy

Q4. Why is SARSA often considered safer than Q-learning in real-world control applications?

- A. SARSA converges faster than Q-learning
- B. SARSA ignores exploration during updates
- C. SARSA updates values using the action actually taken
- D. SARSA requires a model of the environment

Quiz (3/3)

Q5. Why are non-linear function approximators, such as neural networks, used in reinforcement learning?

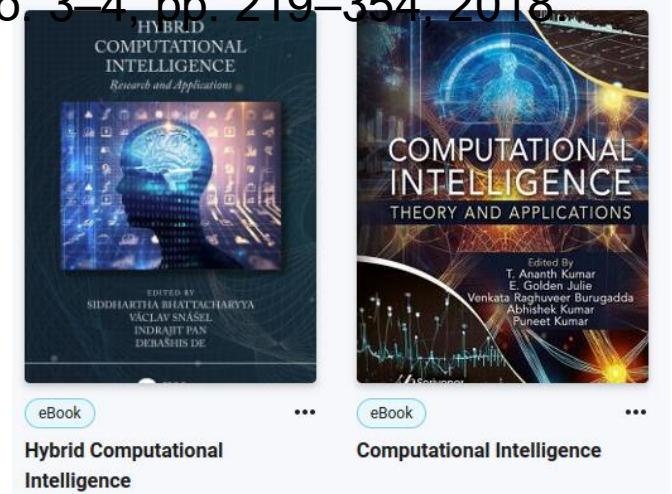
- A. They guarantee convergence to optimal solutions
- B. They eliminate the need for exploration
- C. They can represent complex relationships in high-dimensional spaces
- D. They reduce computational cost

Q6. In the controller gain tuning case study, what role does the reward signal play?

- A. It replaces the controller entirely
- B. It encodes system performance and guides parameter updates
- C. It directly computes optimal gains analytically
- D. It eliminates the need for system feedback

References

- [1] Sutton, R. S., and Barto, A. G., *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, Cambridge, MA, USA, 2018.
- [2] Wang, X., Wang, S., Liang, X., Zhao, D., Huang, J., Xu, X., Dai, B., and Miao, Q., “Deep Reinforcement Learning: A Survey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 5064–5097, 2024.
- [3] Li, Y., “Deep Reinforcement Learning: An Overview,” *arXiv preprint arXiv:1701.07274*, 2017.
- [4] Arulkumaran, K., Deisenroth, M. P., Brundage, M., and Bharath, A. A., “Deep Reinforcement Learning: A Brief Survey,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [5] François-Lavet, V., Henderson, P., Islam, R., Bellemare, M. G., and Pineau, J., “An Introduction to Deep Reinforcement Learning,” *Foundations and Trends in Machine Learning*, vol. 11, no. 3–4, pp. 219–354, 2018.





Nottingham Trent
University

Department of Computer Science

Summary

In this lecture you learnt:

- Reinforcement learning for sequential decision-making
- Agent-environment interaction and MDPs
- States, actions, rewards, and return
- Policies and value functions
- Bellman equations
- Exploration vs exploitation
- Model-free learning (Q-learning, SARSA)
- Deep reinforcement learning
- Policy gradients and actor-critic methods
- RL for control and CI parameter tuning

This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.



Nottingham Trent
University

Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 5: Reinforcement Learning

Dr Pedro Machado pedro.machado@ntu.ac.uk